

MODULE 4

Build a Data-Acquisition System

The Fan Lab: condition monitoring of a rotating machine

TinyML Course · Day 2

Warm-up — four questions from Day 1

1. Train/test split for sensor data: by **window** or by **recording session** — and why?
2. A real **60 Hz** vibration sampled at 100 Hz shows up at what frequency?
3. The data forwarder reports **62 Hz** instead of 100 Hz. Most likely cause?
4. On the nRF52840, where do a model's **weights** live — RAM or flash?

Two minutes with your neighbour, then we compare notes.

Three questions before you touch the hardware

1. **What exactly counts as "failure"** for this machine — and would an expert agree?
2. **Who consumes the alert** — a dashboard, a technician's phone, a maintenance queue?
3. **What does a false alarm cost** vs a missed fault?

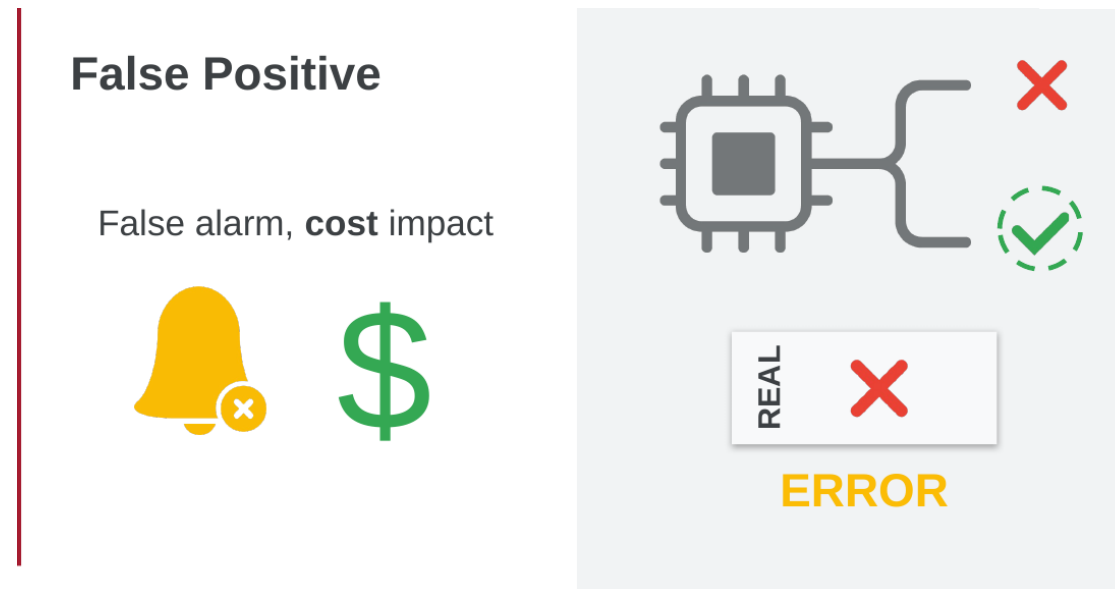


Figure: TinyML on edX (HarvardX), V.J. Reddi et al. — used with permission · Question framing after V.J. Reddi

Day 2 roadmap

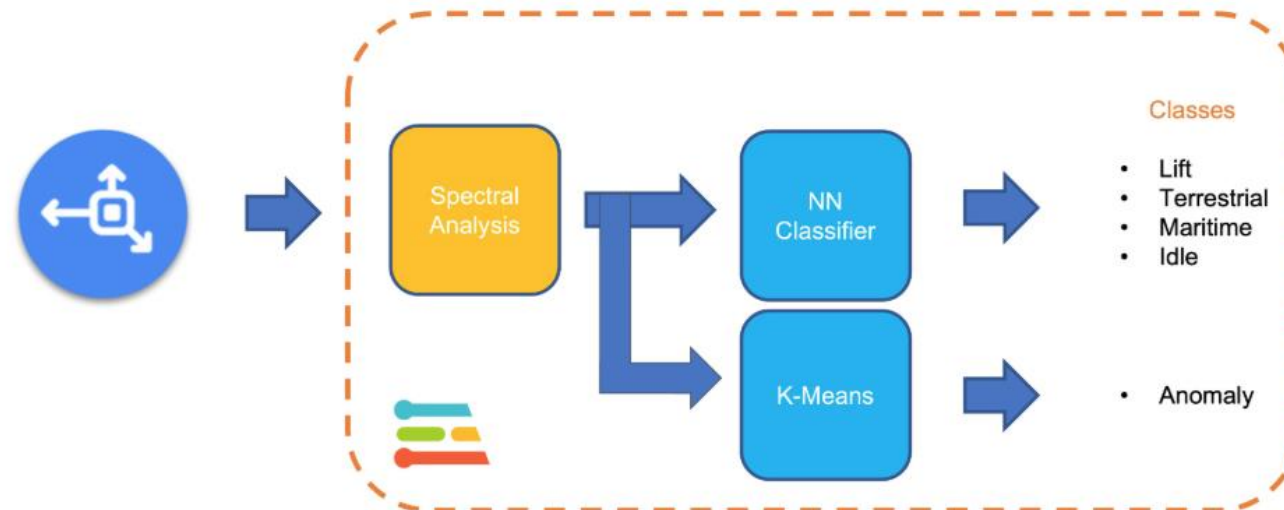
Module	Topic	Output
4 (now)	Fan lab — record a disciplined dataset	Labelled 5-class dataset in Edge Impulse
5	Feature engineering (time + frequency domain)	Features in Python and C
6	ML frameworks for MCUs	emlearn XOR running on your RAK4631

Why condition monitoring?

- Rotating machines (pumps, fans, motors, compressors) **fail in patterns**: imbalance, misalignment, bearing wear, cavitation, blockage
- Classic maintenance strategies:
 - **Run-to-failure** — cheap until it isn't
 - **Scheduled** — replaces healthy parts, misses early failures
 - **Condition-based** — act when the *machine* says so
- Vibration is the richest, cheapest signal for rotating machinery
- TinyML angle: analyse **at the sensor**, transmit only *state*, not raw waveforms

Our target machine: a 120 mm PC fan

- Stand-in for a pump: rotor + bearing + airflow + housing
- 5 V/12 V, ~1000–2500 RPM depending on model and voltage
- Rotation fundamental: **$\text{RPM}/60 \approx 20\text{--}40 \text{ Hz}$** — comfortably inside our accelerometer bandwidth
- Cheap enough to *deliberately break*



Vibration signatures of rotating machines

Fault	Typical signature
Imbalance	Strong sinusoid at $1\times$ rotation frequency, radial
Misalignment	$2\times$ rotation frequency components
Bearing wear	Broadband noise + high-frequency modulated bursts
Rub / scrape	Impulsive, harmonic-rich, often audible
Flow blockage	Changed load $\rightarrow$ RPM shift + reduced broadband level

- Fault physics live in **both** amplitude statistics *and* frequency content
- This is why Module 5 covers time-domain **and** spectral features

The five machine states

Label	How to create it	What physics says
normal	Fan running free, nothing attached	Baseline 1× tone + broadband
imbalance	Tape a small weight (2–4 cm tape blob / M3 nut) to ONE blade	Boosted 1× tone
blocked	Cover intake/exhaust with cardboard ($\geq 80\%$)	RPM shift, reduced flow noise
scrape	Zip-tie / paper edge lightly touching blades	Impulsive, high kurtosis
off	Fan unpowered	Sensor noise floor only

Rule: change ONE thing per state. Same fan, same mounting, same voltage.

Sensor mounting — coupling is everything

- RAK1904 (LIS3DH) sits on the WisBlock base — so we mount **the whole board** on the fan frame
- **Rigid coupling** transmits vibration; soft coupling low-pass filters it away
 - Use screws & nuts
- Mount on the **frame corner**, not on the sticker over the motor hub
- Cable strain relief: tape the USB lead to the desk so it can't rattle

Acquisition parameters

- **Sample rate: 250 Hz** (LIS3DH configured for ± 4 g)
 - Nyquist = 125 Hz $\rightarrow$ captures the 70–80 Hz rotation fundamental
 - Blade-pass frequency (blades $\times$ RPM/60 $\approx$ 150–300 Hz) is **above** Nyquist — we knowingly give it up, or experiment with a higher F_s
- **Window length: 2 s** $\rightarrow$ 200 samples/axis; several rotation periods per window
- **Recording length: 20 s per sample** $\rightarrow$ EI slices into windows later
- ± 4 g range: scraping impacts can exceed ± 2 g; clipping destroys data



Lab step 1 — build the rig (~25 min)

[exercises/module4-fan-dataset/README.md](#) → sections 1–2

- Zip-tie the WisBlock to the fan frame — **rigid coupling**, connector clear
- Flash Module 3's forwarder → verify **≈250 Hz** detected
- Fan on `normal`: watch the live waveform — visible periodicity = good coupling
- **Done when:** forwarder streams clean data with the fan running

Why dataset discipline beats model tuning

- "Garbage in, garbage out" is not a slogan, it's a budget statement:
 - 1 h spent on clean data $\approx$ days saved on debugging mystery accuracy
- Typical silent failure: **the model learns your recording session**, not the machine state
 - Different desk, different USB cable tension, different room $\rightarrow$ different data
- The test set must answer: *"does this work on data the model has never seen?"*
— not: *"can the model memorise Tuesday morning?"*

Discipline rule 1 — repetitions, interleaved

- Record **≥ 5 separate recordings per state**, 20 s each
→ ≥ 100 s/state, ≥ 50 windows/state at 2 s windows
- **Interleave** states: don't record all `normal` then all `imbalance`
 - Cycle: normal → imbalance → blocked → scrape → off → repeat
- Between repetitions of the *same* state: **remount / restart** something
 - Re-stick the tape weight at a slightly different radius
 - Power-cycle the fan
 -

Discipline rule 2 — train/test hygiene

- Edge Impulse default: ~80/20 train/test split — **keep it**
- Split by **recording**, never by window
 - Adjacent 2 s windows from one 20 s recording are near-duplicates
 - If windows from the same recording land in both train *and* test → leakage → fake accuracy
- Practical rule in EI: assign whole *samples* (recordings) to Test, e.g. your last recording of each state
- Never look at test data during Module 5's feature exploration

Discipline rule 3 — metadata

Record it *while you remember it*. Minimum set:

fan model / size: Arctic F12, 120 mm
supply voltage: 12 V (write down if you change it!)
nominal RPM: ~1350 (from datasheet or strobe/audio check)
sensor axis vs shaft: Z axis parallel to rotation axis
mount: 2x zip-tie, frame corner, top-left
weight (imbalance): M3 nut + tape, blade tip
date/desk/person: 2026-07-xx, desk 4, <name>

- In EI: use the **filename/label conventions** — `imbalance.desk4.01`
- Keep a `NOTES.md` next to your project — future-you is a different person

Ingestion recap — the Module 3 forwarder

- We reuse the **accel-forwarder firmware from Module 3** unchanged (LIS3DH @ 250 Hz → `x\ty\tz` lines on USB serial @ 115200)
- On the laptop:

```
edge-impulse-data-forwarder
```

```
# → login, select project, name axes: accX, accY, accZ
```

- Forwarder auto-detects the sample rate from line timing — verify it reports **≈ 250 Hz**
- Then: EI Studio → **Data acquisition** → device shows up → set label + length → *Start sampling*

Recording in Edge Impulse Studio



- Dashboard
- Devices
- Data acquisition
- Experiments
- EON Tuner
- Impulse #3 CNN
 - Create impulse
 - Raw data
 - Classifier
 - Retrain model
 - Live classification
 - Model testing
 - Deployment
- Versioning

Morten / fan-condition-monitoring PERSONAL

Target: Arduino Nicla Vis...

M

Dataset

Data explorer

Data sources

Synthetic data

AI labeling

CSV Wizard

DATA COLLECTED
8m 27s



DATASET SPLIT
77% / 23%



Collect data



[Connect a device](#) to start building your dataset.

Dataset



Training (20) Test (6)



SAMPLE NAME	LABEL	ADDED	LENGTH	
imbalance.6vgfh26j	imbalance	Aug 09 2026, 19...	20s	
imbalance.6vgffv1n	imbalance	Aug 09 2026, 19...	20s	
imbalance.6vgfd3tj	imbalance	Aug 09 2026, 19...	20s	
imbalance.6vgfc5du	imbalance	Aug 09 2026, 19...	20s	
scrape.6vgf8c7b	scrape	Aug 09 2026, 19...	20s	
scrape.6vgf5nk0	scrape	Aug 09 2026, 19...	20s	
scrape.6vgf33f3	scrape	Aug 09 2026, 19...	20s	

RAW DATA

Click on a sample to load...



Lab step 2 — record the dataset (~60 min)

[exercises/module4-fan-dataset/README.md](#) → sections 3–5

- 5 states × 4 interleaved rounds × 20 s = **~27 min of machine time**
- Label in Studio exactly as the state table says; metadata per the template
- Run the two sanity checks (next slides) **as you go**, not at the end
- **Done when:** the 12-item dataset-quality checklist is all



Sanity check 1 — look at the raw data

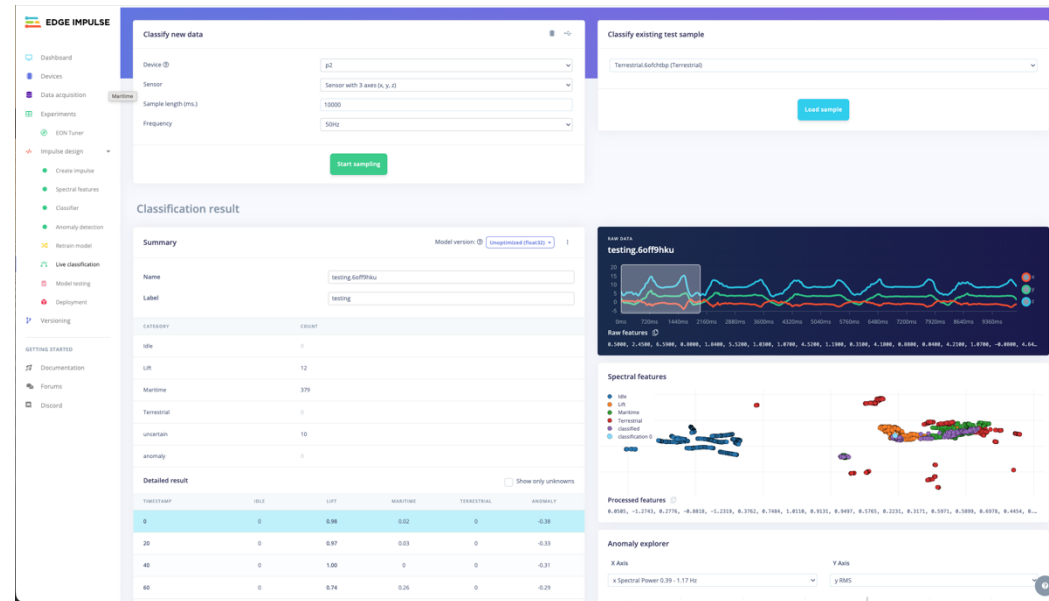
For every state, click a sample in EI and **eyeball the waveform**:

- **off** → flat lines ($\pm$ noise $\sim$ a few mg)
- **normal** → periodic ripple, stable amplitude
- **imbalance** → clearly larger periodic ripple
- **scrape** → spiky, irregular
- **blocked** → similar to normal but different level (be sceptical!)

If two states look identical → make the physical fault stronger *now*, not on Day 3.

Sanity check 2 — the feature explorer

- Quick impulse: **Spectral Analysis** block on 2 s windows → **Generate features**
- The **feature explorer** projects all windows to 3-D (UMAP-style)
- What you want: 5 visibly distinct clusters (off will be far away)
- What clusters *touching* means: those two states will confuse the classifier



Common failure modes (and fixes)

Symptom	Cause	Fix
Forwarder reports wrong Hz	<code>delay()</code> -based loop timing	timer-driven sampling (Module 3 code)
All classes overlap in explorer	Soft mounting	zip-tie tighter, remount
<code>normal</code> forms 2 clusters	Mount changed mid-session	re-record one cluster's files
Clipped waveform at ± 4 g	Scrape too aggressive	lighter touch
One class much fewer windows	Uneven recording	top up recordings
Perfect separation incl. blocked	Might be genuinely good — or leakage	check session interleaving

Optional extension — parallel audio

- RAK18000 PDM mic (IO slot, DATA `WB_IO3` , CLK `WB_IO4`) can record the same states **acoustically**
- Audio at 16 kHz captures what 100 Hz vibration cannot: blade-pass, flow turbulence, bearing hiss
- Workflow (from Module 3): PCM dump → WAV → `edge-impulse-uploader`
- **Do this only if your vibration dataset is done and clean**

Exercise hand-off — the fan lab

[exercises/module4-fan-dataset/README.md](#)

1. Build the rig: mount WisBlock on fan (photo checklist)
 2. Verify the forwarder path (Module 3 firmware, 100 Hz)
 3. Define the 5 states — physically prepare each fault
 4. Record: $\geq 5 \times 20$ s per state, interleaved, metadata as you go
 5. Label + train/test split in EI Studio
 6. Feature-explorer sanity check
 7. Run the **dataset-quality checklist** — all boxes ticked before lunch
- 🕒 Budget: ~2.5 h · Work in pairs, but **each project needs its own dataset**

Module 4 checkpoint

You now have:

- ✓
 - A physical condition-monitoring rig (sensor rigidly mounted on the machine)
- ✓
 - Five defined, reproducible machine states
- ✓
 - ≥ 250 labelled windows in Edge Impulse, split cleanly into train/test
- ✓
 - Metadata that lets anyone reproduce the recording
- ✓
 - Visual evidence (feature explorer) that the classes are separable

Next (Module 5): open the black box — *what* makes these classes separable, computed by hand in Python and then in C on the nRF52840.